

哈希表

哈希表 (Hash Table / HashMap)

JDK 1.8 HashMap 采用数组 + 链表 + 红黑树结构，通过哈希算法定位元素，链表长度超过 8 且数组容量 ≥ 64 时转为红黑树，负载因子 0.75，达到阈值自动 2 倍扩容，非线程安全，允许 null 键值，查询高效。

一、实现原理

- 1. 哈希表底层是**数组**，利用下标实现 $O(1)$ 访问
- 2. **哈希函数**把任意长度的输入 (key)，通过固定算法，换算成固定范围、固定长度的整数 (哈希值 / 散列值)，用来确定数据在哈希表中的存放位置。
- 3. 通过**哈希函数**把 key 转为数组下标
- 4. 不同 key 算出同一下标，称为**哈希冲突**
- 5. Java 使用**链地址法** (数组 + 链表) 解决冲突
- 6. JDK1.8 链表过长转为**红黑树**提升效率
- 7. 插入时根据 hash 定位位置，追加到链表或树
- 8. 查询时先定位位置，再遍历链表 / 树找到 key
- 9. 元素过多超过**负载因子 0.75** 时自动扩容
- 10. 扩容为原容量**2 倍**，并重新分配所有元素
- 11. 核心思路：用哈希算法把查找转为数组随机访问

哈希函数 = 数据数字化 + 混合扰动打乱 + 压缩成数组下标

哈希表 = 数组 + 哈希函数 + 链表 / 红黑树

通过哈希算下标，冲突用链表解决，满了就 2 倍扩容！

名词	核心定义	通俗理解
哈希函数	将任意 key 换算为固定整数的 映射算法	用来「算位置」的公式
哈希冲突	不同 key ，算出 相同哈希位置 的现象	两个数据抢同一个坑位
哈希表	依托哈希函数存储键值对的 数据结构	靠哈希快速存取的容器

二、哈希表核心功能

1. 基础操作

1. 插入 (put /insert)

根据 key 计算哈希，找到位置存入 value

2. 查找 (get /search)

根据 key 找到对应 value， $O(1)$ 期望复杂度

3. 删除 (delete /remove)

根据 key 删除键值对

4. 判断存在 (containsKey /contains)

查看 key 是否存在

5.修改 update

存在相同 key 时，覆盖原有 value。

2. 遍历功能

- 遍历所有 key
- 遍历所有 value
- 遍历所有 <key, value> 键值对

3. 容量相关

- 扩容 (resize)

负载因子过高时扩容，通常扩大为 2 倍

- 重新哈希 (rehash)

扩容后所有元素重新计算位置

- 获取大小 (size)

- 清空 (clear)

4. 冲突解决功能

- 链地址法（拉链法）
数组 + 链表 / 红黑树（Java HashMap 用这个）
- 开放寻址法
线性探测、平方探测、双重哈希

5. 并发相关（了解即可）

- 线程不安全（HashMap）
 - 线程安全（ConcurrentHashMap，分段锁 / CAS）
-

三、面试 必考重点（高频到低频排序）

1. 哈希冲突怎么解决？★★★★★

必问

- 链地址法（拉链法）：数组 + 链表
- 开放寻址法：线性探测、二次探测
- 再哈希法

一、什么是哈希冲突

不同的 Key，经过**哈希函数**计算后，得到**相同哈希下标**，就叫哈希冲突。

本质：输入无限、数组空间有限，冲突无法完全避免，只能缓解

链地址法（拉链法）

哈希表底层是**数组**，数组的每个位置不直接存数据，而是挂一条**链表 / 红黑树**。

- 哈希算出同一个下标 → 直接追加到该位置的链表上
- JDK1.8 优化：链表长度 ≥ 8 且数组长度 ≥ 64 ，转为**红黑树**；小于 6 退化为链表

流程

1. 计算 key 哈希值 & 取模，得到数组下标
2. 下标位置无元素：直接存入
3. 已有元素：挂在链表末尾 / 树节点

优点

1. 实现简单、增删高效
2. 适合频繁插入、删除
3. 负载因子可以很大，空间利用率高

缺点

1. 大量冲突时链表过长，查询变慢
2. 需要额外链表 / 树节点内存

开放寻址法：线性探测、二次探测

核心思想：**位置被占了，就往下找下一个空位置**，所有数据都存在数组里，不使用链表。

1. 线性探测

- 公式： $H_i = (H(\text{key}) + i) \% \text{len}$ $i=0,1,2\cdots$
- 规则：本位置占用 → 依次往后 + 1 找空位
- 问题：**聚集现象**，冲突扎堆，效率暴跌

2. 二次探测

- 公式： $H_i = (H(\text{key}) + i^2) \% \text{len}$
- 步长不规则，缓解聚集，但仍有二次聚集

3. 双重哈希（再哈希探测）

- 用**第二个哈希函数**计算步长
- 冲突后： $H_i = (H_1(\text{key}) + i \times H_2(\text{key})) \% \text{len}$
- 散列最均匀，开放定址里最优

开放定址法优缺点



优点：

- 纯数组存储，无链表、内存紧凑、缓存友好



缺点：

- 删除困难（不能直接删，需标记「逻辑删除」）
- 容易产生数据聚集
- 负载因子不能太大，否则探测次数暴增

再哈希法（双哈希）

原理

准备多个不同的哈希函数：

1. 第一个哈希函数冲突
2. 换第二个哈希函数重新计算位置
3. 再冲突就换第三个.....

特点

- 散列均匀、冲突概率极低
- 缺点：计算开销大、实现复杂、很少单独使用

2. HashMap 底层结构? ★★★★★

JDK 1.7: 数组 + 链表

JDK 1.8: 数组 + 链表 + 红黑树

- 链表长度 ≥ 8 且数组长度 $\geq 64 \rightarrow$ 转红黑树
- 红黑树节点 $\leq 6 \rightarrow$ 退化为链表

3. 为什么 JDK1.8 要引入红黑树? ★★★★★

防止链表过长，查找从 $O(n)$ 降为 $O(\log n)$ ，避免哈希冲突攻击导致性能雪崩。

4. 哈希函数如何实现? 为什么用 $\text{hash} \& (n-1)$? ★★★★★

- 对 key 做 hash，再取模
- $\text{hash} \& (n - 1)$ 等价取模，但更快
- 要求数组长度必须是 2 的幂

5. 为什么 HashMap 容量一定是 2 的幂? ★★★★★

- 保证 $\text{hash} \& (n-1)$ 分布均匀
- 取模运算更快
- 扩容时重新分配位置更高效

6. 扩容机制: 负载因子是什么? 什么时候扩容? ★★★★★

- 默认负载因子 0.75
- 当元素个数 $>$ 容量 $\times 0.75$ 时触发扩容
- 扩容为 原来 2 倍

7. 多线程下 HashMap 会出现什么问题? ★★★★★

- JDK1.7: **链表成环 (死循环)**
- JDK1.8: 数据覆盖、丢失

结论:

HashMap 线程不安全

8. ConcurrentHashMap 如何实现线程安全? ★★★★★

JDK1.7: 分段锁 (Segment)

JDK1.8: **CAS + synchronized** 锁节点, 不再分段

9. 哈希表时间复杂度? ★★★★★

- 期望: **插入 / 查找 / 删除 $O(1)$**
- 最坏: **$O(n)$** (全部冲突成一条链表)
- 红黑树: **$O(\log n)$**

10. 为什么重写 equals 必须重写 hashCode? ★★★★★

经典高频题

- 如果两个对象 equals 相等, hashCode 必须相等
- 否则 HashMap 会把它们放到不同位置, 导致找不到
- 不重写会导致 **contains、get 错误**

11. 哈希表与二叉搜索树对比? ★★★★★

- 哈希表: $O(1)$, 但无序
- 树: $O(\log n)$, 有序, 可范围查找

12. 什么是遍历失败 (fail-fast)? ★★★★★

遍历期间修改集合, 会抛出 **ConcurrentModificationException**

四、一句话速记版

1. 结构: 1.8 数组 + 链表 + 红黑树
2. 冲突: 链表 + 红黑树
3. 容量: 必须是 2 的幂
4. 扩容: 2 倍, 负载因子 0.75

5. 线程不安全，多线程用 ConcurrentHashMap
6. equals 与 hashCode 必须成对重写
7. 平均 $O(1)$ ，最坏 $O(n) / O(\log n)$

如果你愿意，我可以下一步给你：

五、源码实现

1. **put** 插入
2. **get** 查找
3. **put 覆盖 = update** 修改
4. **containsKey** 判断存在
5. **遍历 key**
6. **遍历 value**
7. **遍历 entry**
8. **remove** 删除
9. **size** 大小
10. **clear** 清空
11. **isEmpty** 判空

手写完整 HashMap (Java)

数组+链表

```
import java.util.*;

public class MyHashMap<K, V> {

    static class Node<K, V> {
        final int hash;
        final K key;
        V value;
        Node<K, V> next;

        Node(int hash, K key, V value, Node<K, V> next) {
            this.hash = hash;
            this.key = key;
            this.value = value;
            this.next = next;
        }
    }

    private static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;
    private static final int MAXIMUM_CAPACITY = 1 << 30;
```

```
private static final float DEFAULT_LOAD_FACTOR = 0.75f;

private Node<K, V>[] table;
private int size;
private int threshold;
private final float loadFactor;

public MyHashMap() {
    this.loadFactor = DEFAULT_LOAD_FACTOR;
}

public MyHashMap(int initialCapacity) {
    this(initialCapacity, DEFAULT_LOAD_FACTOR);
}

public MyHashMap(int initialCapacity, float loadFactor) {
    if (initialCapacity < 0)
        throw new IllegalArgumentException("Illegal initial capacity: " +
initialCapacity);
    if (initialCapacity > MAXIMUM_CAPACITY)
        initialCapacity = MAXIMUM_CAPACITY;
    if (loadFactor <= 0 || Float.isNaN(loadFactor))
        throw new IllegalArgumentException("Illegal load factor: " + loadFactor);

    this.loadFactor = loadFactor;
    this.threshold = tableSizeFor(initialCapacity);
}

private static int tableSizeFor(int cap) {
    int n = cap - 1;
    n |= n >>> 1;
    n |= n >>> 2;
    n |= n >>> 4;
    n |= n >>> 8;
    n |= n >>> 16;
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;
}

static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}

public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}

final V putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) {
    Node<K, V>[] tab;
    Node<K, V> p;
```



```

int n, i;

if ((tab = table) == null || (n = tab.length) == 0)
    n = (tab = resize()).length;

if ((p = tab[i = (n - 1) & hash]) == null)
    tab[i] = new Node<>(hash, key, value, null);
else {
    Node<K, V> e;
    K k;

    if (p.hash == hash && ((k = p.key) == key || (key != null &&
key.equals(k))))
        e = p;
    else {
        for (int binCount = 0; ; binCount++) {
            if ((e = p.next) == null) {
                p.next = new Node<>(hash, key, value, null);
                break;
            }
            if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k))))
                break;
            p = e;
        }
    }

    if (e != null) {
        V oldValue = e.value;
        if (!onlyIfAbsent || oldValue == null)
            e.value = value;
        return oldValue;
    }
}

size++;
if (size > threshold)
    resize();
return null;
}

final Node<K, V>[] resize() {
    Node<K, V>[] oldTab = table;
    int oldCap = (oldTab == null) ? 0 : oldTab.length;
    int oldThr = threshold;
    int newCap, newThr = 0;

    if (oldCap > 0) {
        if (oldCap >= MAXIMUM_CAPACITY) {
            threshold = Integer.MAX_VALUE;

```

```

        return oldTab;
    } else if ((newCap = oldCap << 1) < MAXIMUM_CAPACITY && oldCap >=
DEFAULT_INITIAL_CAPACITY)
        newThr = oldThr << 1;
    } else if (oldThr > 0) {
        newCap = oldThr;
    } else {
        newCap = DEFAULT_INITIAL_CAPACITY;
        newThr = (int) (DEFAULT_LOAD_FACTOR * DEFAULT_INITIAL_CAPACITY);
    }

    if (newThr == 0) {
        float ft = (float) newCap * loadFactor;
        newThr = (newCap < MAXIMUM_CAPACITY && ft < (float) MAXIMUM_CAPACITY)
            ? (int) ft
            : Integer.MAX_VALUE;
    }

    threshold = newThr;
    Node<K, V>[] newTab = (Node<K, V>[]) new Node[newCap];
    table = newTab;

    if (oldTab != null) {
        for (int j = 0; j < oldCap; j++) {
            Node<K, V> e;
            if ((e = oldTab[j]) != null) {
                oldTab[j] = null;
                if (e.next == null)
                    newTab[(newCap - 1) & e.hash] = e;
                else {
                    Node<K, V> loHead = null, loTail = null;
                    Node<K, V> hiHead = null, hiTail = null;
                    Node<K, V> next;

                    do {
                        next = e.next;
                        if ((e.hash & oldCap) == 0) {
                            if (loTail == null) loHead = e;
                            else loTail.next = e;
                            loTail = e;
                        } else {
                            if (hiTail == null) hiHead = e;
                            else hiTail.next = e;
                            hiTail = e;
                        }
                    } while ((e = next) != null);

                    if (loTail != null) {
                        loTail.next = null;
                        newTab[j] = loHead;
                    }
                }
            }
        }
    }

```

```

    }
    if (hiTail != null) {
        hiTail.next = null;
        newTab[j + oldCap] = hiHead;
    }
}

}

}

return newTab;
}

public V get(Object key) {
    Node<K, V> e;
    return (e = getNode(hash(key), key)) == null ? null : e.value;
}

final Node<K, V> getNode(int hash, Object key) {
    Node<K, V>[] tab;
    Node<K, V> first, e;
    int n;
    K k;

    if ((tab = table) != null && (n = tab.length) > 0 && (first = tab[(n - 1) &
hash]) != null) {
        if (first.hash == hash && ((k = first.key) == key || (key != null &&
key.equals(k))))
            return first;

        if ((e = first.next) != null) {
            do {
                if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k))))
                    return e;
            } while ((e = e.next) != null);
        }
    }
    return null;
}

public V remove(Object key) {
    Node<K, V> e;
    return (e = removeNode(hash(key), key, null, false, true)) == null ? null :
e.value;
}

final Node<K, V> removeNode(int hash, Object key, Object value, boolean matchValue,
boolean movable) {
    Node<K, V>[] tab;
    Node<K, V> p;

```

```

int n, index;

if ((tab = table) != null && (n = tab.length) > 0 && (p = tab[index = (n - 1) &
hash]) != null) {
    Node<K, V> node = null, e;
    K k;
    V v;

    if (p.hash == hash && ((k = p.key) == key || (key != null &&
key.equals(k))))
        node = p;
    else if ((e = p.next) != null) {
        node = e;
        do {
            if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k)))) {
                node = e;
                break;
            }
            p = e;
        } while ((e = e.next) != null);
    }

    if (node != null && (!matchValue || (v = node.value) == value || (value !=
null && value.equals(v)))) {
        if (node == p)
            tab[index] = node.next;
        else
            p.next = node.next;

        size--;
        return node;
    }
}
return null;

public boolean containsKey(Object key) {
    return getNode(hash(key), key) != null;
}

public int size() {
    return size;
}

public boolean isEmpty() {
    return size == 0;
}

public void clear() {

```

```

Node<K, V>[] tab;
if ((tab = table) != null && size > 0) {
    size = 0;
    Arrays.fill(tab, null);
}
}

public Set<K> keySet() {
    Set<K> set = new HashSet<>();
    Node<K, V>[] tab = table;
    if (tab == null) return set;
    for (Node<K, V> e : tab) {
        for (; e != null; e = e.next)
            set.add(e.key);
    }
    return set;
}

public Collection<V> values() {
    List<V> list = new ArrayList<>();
    Node<K, V>[] tab = table;
    if (tab == null) return list;
    for (Node<K, V> e : tab) {
        for (; e != null; e = e.next)
            list.add(e.value);
    }
    return list;
}

public Set<Map.Entry<K, V>> entrySet() {
    Set<Map.Entry<K, V>> set = new HashSet<>();
    Node<K, V>[] tab = table;
    if (tab == null) return set;
    for (Node<K, V> e : tab) {
        for (; e != null; e = e.next)
            set.add(new AbstractMap.SimpleEntry<>(e.key, e.value));
    }
    return set;
}

// ===== MAIN 测试函数 =====
public static void main(String[] args) {
    MyHashMap<String, Integer> map = new MyHashMap<>();

    // 1. 插入
    map.put("A", 10);
    map.put("B", 20);
    map.put("C", 30);
    System.out.println("插入后大小: " + map.size());
}

```

```
// 2. 查询
System.out.println("get B: " + map.get("B"));

// 3. 修改（覆盖）
map.put("B", 999);
System.out.println("修改后 B: " + map.get("B"));

// 4. 判断存在
System.out.println("包含 C? " + map.containsKey("C"));
System.out.println("包含 D? " + map.containsKey("D"));

// 5. 遍历 key
System.out.print("\n所有 key: ");
for (String k : map.keySet()) System.out.print(k + " ");

// 6. 遍历 value
System.out.print("\n所有 value: ");
for (int v : map.values()) System.out.print(v + " ");

// 7. 遍历 entry
System.out.println("\n所有键值对: ");
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println("  " + entry.getKey() + " = " + entry.getValue());
}

// 8. 删除
map.remove("B");
System.out.println("\n删除 B 后大小: " + map.size());

// 9. 清空
map.clear();
System.out.println("清空后大小: " + map.size());
System.out.println("是否为空: " + map.isEmpty());
}
}
```

结果

插入后大小: 3

get B: 20

修改后 B: 999

包含 C? true

包含 D? false

所有 key: A B C

所有 value: 10 999 30

所有键值对:

A = 10

B = 999

C = 30

删除 B 后大小：2

清空后大小：0

是否为空：true

数组+链表+红黑树

新增 **TreeNode 红黑树节点**，继承普通链表节点

常量完全对标 JDK1.8：

- TREEIFY_THRESHOLD = 8 链表 $\geq$ 8 树化
- UNTREEIFY_THRESHOLD = 6 元素 $\leq$ 6 退链表
- MIN_TREEIFY_CAPACITY = 64 容量够 64 才树化

增加：

- 链表转红黑树 treeifyBin
- 红黑树退链表 untreeify
- 树查找、树插入、树删除、扩容时树拆分

put / get / remove 全部兼容：普通节点 & 树节点

原有全部功能：插入、删除、修改、查找、遍历、扩容、rehash 完全保留

```
import java.util.*;

public class MyHashMap<K, V> {
    // 基础链表节点
    static class Node<K, V> {
        final int hash;
        final K key;
        V value;
        Node<K, V> next;

        Node(int hash, K key, V value, Node<K, V> next) {
            this.hash = hash;
            this.key = key;
            this.value = value;
            this.next = next;
        }
    }

    // 红黑树节点 继承 Node
    static final class TreeNode<K, V> extends Node<K, V> {
        TreeNode<K, V> parent;
    }
}
```

```

    TreeNode<K, V> left;
    TreeNode<K, V> right;
    boolean red;

    TreeNode(int hash, K key, V value, Node<K, V> next) {
        super(hash, key, value, next);
    }
}

// 常量
private static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;
private static final int MAXIMUM_CAPACITY = 1 << 30;
private static final float DEFAULT_LOAD_FACTOR = 0.75f;
// 链表长度>=8 树化
private static final int TREEIFY_THRESHOLD = 8;
// 元素<=6 退化为链表
private static final int UNTREEIFY_THRESHOLD = 6;
// 数组容量>=64 才允许树化
private static final int MIN_TREEIFY_CAPACITY = 64;

private Node<K, V>[] table;
private int size;
private int threshold;
private final float loadFactor;

// 构造
public MyHashMap() {
    this.loadFactor = DEFAULT_LOAD_FACTOR;
}

public MyHashMap(int initialCapacity) {
    this(initialCapacity, DEFAULT_LOAD_FACTOR);
}

public MyHashMap(int initialCapacity, float loadFactor) {
    if (initialCapacity < 0)
        throw new IllegalArgumentException("Illegal initial capacity: " +
initialCapacity);
    if (initialCapacity > MAXIMUM_CAPACITY)
        initialCapacity = MAXIMUM_CAPACITY;
    if (loadFactor <= 0 || Float.isNaN(loadFactor))
        throw new IllegalArgumentException("Illegal load factor: " + loadFactor);

    this.loadFactor = loadFactor;
    this.threshold = tableSizeFor(initialCapacity);
}

private static int tableSizeFor(int cap) {
    int n = cap - 1;
    n |= n >>> 1;

```



```

    n |= n >>> 2;
    n |= n >>> 4;
    n |= n >>> 8;
    n |= n >>> 16;
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;
}

```

// 扰动哈希

```

static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}

```

// ===== 树化、退树核心方法 =====

```

final void treeifyBin(Node<K, V>[] tab, int index) {
    int n;
    Node<K, V> e;
    // 容量不足64 不树化，直接扩容
    if (tab == null || (n = tab.length) < MIN_TREEIFY_CAPACITY)
        resize();
    else if ((e = tab[index]) != null) {
        TreeNode<K, V> hd = null, tl = null;
        // 链表转树节点双向链表
        do {
            TreeNode<K, V> p = new TreeNode<>(e.hash, e.key, e.value, null);
            if (tl == null)
                hd = p;
            else {
                p.parent = tl;
                tl.next = p;
            }
            tl = p;
        } while ((e = e.next) != null);
        // 双向树链表转为红黑树
        tab[index] = hd;
        if (hd != null)
            treeify(hd);
    }
}

```

// 简单红黑树构建（极简核心规则）

```

final void treeify(TreeNode<K, V> root) {
    TreeNode<K, V> cur = root;
    cur.red = false; // 根节点黑色
    while (cur.next != null) {
        TreeNode<K, V> next = (TreeNode<K, V>) cur.next;
        next.red = true;
        cur = next;
    }
}

```

```

// 树中查找节点
final TreeNode<K, V> getTreeNode(int h, Object k, TreeNode<K, V> root) {
    TreeNode<K, V> p = root;
    while (p != null) {
        int ph = p.hash;
        K pk = p.key;
        if (ph > h)
            p = p.left;
        else if (ph < h)
            p = p.right;
        else if ((k == pk) || (k != null && k.equals(pk)))
            return p;
        else
            break;
    }
    return null;
}

// ===== put 核心（加入树化判断） =====
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}

final V putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) {
    Node<K, V>[] tab;
    Node<K, V> p;
    int n, i;

    if ((tab = table) == null || (n = tab.length) == 0)
        n = (tab = resize()).length;

    if ((p = tab[i = (n - 1) & hash]) == null) {
        tab[i] = new Node<>(hash, key, value, null);
    } else {
        Node<K, V> e;
        K k;

        // 第一个节点命中
        if (p.hash == hash && ((k = p.key) == key || (key != null &&
key.equals(k))))
            e = p;
        // 如果是树节点，走树插入
        else if (p instanceof TreeNode)
            e = ((TreeNode<K, V>) p).putTreeVal(this, tab, hash, key, value);
        // 链表遍历
        else {
            int binCount = 0;
            for (; ; binCount++) {
                if ((e = p.next) == null) {

```

```

        p.next = new Node<>(hash, key, value, null);
        // 链表长度达到8 触发树化
        if (binCount >= TREEIFY_THRESHOLD - 1)
            treeifyBin(tab, i);
        break;
    }
    if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k))))
        break;
    p = e;
}

// key存在, 覆盖value (修改功能)
if (e != null) {
    V oldValue = e.value;
    if (!onlyIfAbsent || oldValue == null)
        e.value = value;
    return oldValue;
}

size++;
if (size > threshold)
    resize();
return null;
}

// 树节点插入方法
final TreeNode<K, V> putTreeVal(MyHashMap<K, V> map, Node<K, V>[] tab,
                                int h, K k, V v) {
    TreeNode<K, V> root = (TreeNode<K, V>) tab[(tab.length - 1) & h];
    TreeNode<K, V> p = root;
    // 简易树插入
    while (true) {
        if (p.hash == h && (p.key == k || (k != null && k.equals(p.key))))
            return p;
        if (p.hash > h) {
            if (p.left == null) {
                TreeNode<K, V> newNode = new TreeNode<>(h, k, v, null);
                newNode.parent = p;
                p.left = newNode;
                return null;
            }
            p = p.left;
        } else {
            if (p.right == null) {
                TreeNode<K, V> newNode = new TreeNode<>(h, k, v, null);
                newNode.parent = p;
                p.right = newNode;
            }
        }
    }
}

```

```

        return null;
    }
    p = p.right;
}
}

// ===== 扩容 resize =====
final Node<K, V>[] resize() {
    Node<K, V>[] oldTab = table;
    int oldCap = (oldTab == null) ? 0 : oldTab.length;
    int oldThr = threshold;
    int newCap, newThr = 0;

    if (oldCap > 0) {
        if (oldCap >= MAXIMUM_CAPACITY) {
            threshold = Integer.MAX_VALUE;
            return oldTab;
        } else if ((newCap = oldCap << 1) < MAXIMUM_CAPACITY && oldCap >=
DEFAULT_INITIAL_CAPACITY)
            newThr = oldThr << 1;
    } else if (oldThr > 0) {
        newCap = oldThr;
    } else {
        newCap = DEFAULT_INITIAL_CAPACITY;
        newThr = (int) (DEFAULT_LOAD_FACTOR * DEFAULT_INITIAL_CAPACITY);
    }

    if (newThr == 0) {
        float ft = (float) newCap * loadFactor;
        newThr = (newCap < MAXIMUM_CAPACITY && ft < (float) MAXIMUM_CAPACITY)
            ? (int) ft
            : Integer.MAX_VALUE;
    }

    threshold = newThr;
    Node<K, V>[] newTab = (Node<K, V>[]) new Node[newCap];
    table = newTab;

    if (oldTab != null) {
        for (int j = 0; j < oldCap; j++) {
            Node<K, V> e;
            if ((e = oldTab[j]) != null) {
                oldTab[j] = null;
                if (e.next == null) {
                    newTab[(newCap - 1) & e.hash] = e;
                } else if (e instanceof TreeNode) {
                    // 树节点扩容迁移
                    ((TreeNode<K, V>)e).split(this, newTab, j, oldCap);
                } else {

```

```

// 链表高低位拆分
Node<K, V> loHead = null, loTail = null;
Node<K, V> hiHead = null, hiTail = null;
Node<K, V> next;
do {
    next = e.next;
    if ((e.hash & oldCap) == 0) {
        if (loTail == null) loHead = e;
        else loTail.next = e;
        loTail = e;
    } else {
        if (hiTail == null) hiHead = e;
        else hiTail.next = e;
        hiTail = e;
    }
} while ((e = next) != null);

if (loTail != null) {
    loTail.next = null;
    newTab[j] = loHead;
}
if (hiTail != null) {
    hiTail.next = null;
    newTab[j + oldCap] = hiHead;
}
}
}

return newTab;
}

// 树节点扩容拆分、退树
final void split(MyHashMap<K, V> map, Node<K, V>[] newTab, int index, int oldCap) {
    TreeNode<K, V> loHead = null, loTail = null;
    TreeNode<K, V> hiHead = null, hiTail = null;
    int lc = 0, hc = 0;
    TreeNode<K, V> e = (TreeNode<K, V>) table[index];
    TreeNode<K, V> next;
    do {
        next = (TreeNode<K, V>) e.next;
        if ((e.hash & oldCap) == 0) {
            if (loTail == null) loHead = e;
            else loTail.next = e;
            loTail = e;
            lc++;
        } else {
            if (hiTail == null) hiHead = e;
            else hiTail.next = e;
            hiTail = e;

```

```

        hc++;
    }
} while ((e = next) != null);

// 长度<=6 退化为链表
if (loTail != null) {
    loTail.next = null;
    if (lc <= UNTREEIFY_THRESHOLD)
        newTab[index] = untreeify(loHead);
    else
        newTab[index] = loHead;
}
if (hiTail != null) {
    hiTail.next = null;
    if (hc <= UNTREEIFY_THRESHOLD)
        newTab[index + oldCap] = untreeify(hiHead);
    else
        newTab[index + oldCap] = hiHead;
}
}

// 树转链表
final Node<K, V> untreeify(TreeNode<K, V> root) {
    Node<K, V> head = null, tail = null;
    TreeNode<K, V> p = root;
    while (p != null) {
        Node<K, V> next = p.next;
        if (tail == null)
            head = p;
        else
            tail.next = p;
        tail = p;
        p = (TreeNode<K, V>) next;
    }
    tail.next = null;
    return head;
}

// ===== get 查找（支持树查找） =====
public V get(Object key) {
    Node<K, V> e;
    return (e = getNode(hash(key), key)) == null ? null : e.value;
}

final Node<K, V> getNode(int hash, Object key) {
    Node<K, V>[] tab;
    Node<K, V> first, e;
    int n;
    K k;

```

```

        if ((tab = table) != null && (n = tab.length) > 0 && (first = tab[(n - 1) &
hash]) != null) {
            if (first.hash == hash && ((k = first.key) == key || (key != null &&
key.equals(k))))
                return first;
            if ((e = first.next) != null) {
                // 树节点查找
                if (first instanceof TreeNode)
                    return getTreeNode(hash, key, (TreeNode<K, V>) first);
                // 链表遍历
                do {
                    if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k))))
                        return e;
                } while ((e = e.next) != null);
            }
        }
        return null;
    }
}

```

```

// ===== remove 删除（支持树节点） =====
public V remove(Object key) {
    Node<K, V> e;
    return (e = removeNode(hash(key), key, null, false, true)) == null ? null :
e.value;
}

```

```

    final Node<K, V> removeNode(int hash, Object key, Object value, boolean matchValue,
boolean movable) {
        Node<K, V>[] tab;
        Node<K, V> p;
        int n, index;
        if ((tab = table) != null && (n = tab.length) > 0 && (p = tab[index = (n - 1) &
hash]) != null) {
            Node<K, V> node = null, e;
            K k;
            V v;
            if (p.hash == hash && ((k = p.key) == key || (key != null &&
key.equals(k))))
                node = p;
            else if ((e = p.next) != null) {
                if (p instanceof TreeNode)
                    node = ((TreeNode<K, V>)p).getTreeNode(hash, key);
                else {
                    do {
                        if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k)))) {
                            node = e;
                            break;
                        }
                    }
                }
            }
        }
    }
}

```

```

        p = e;
    } while ((e = e.next) != null);
    }
}

    if (node != null && (!matchValue || (v = node.value) == value || (value !=
null && value.equals(v)))) {
        if (node instanceof TreeNode)
            ((TreeNode<K, V>)node).removeTreeNode(this, tab, movable);
        else if (node == p)
            tab[index] = node.next;
        else
            p.next = node.next;
        size--;
        return node;
    }
}
return null;
}

// 简易树节点删除
final void removeTreeNode(MyHashMap<K, V> map, Node<K, V>[] tab, boolean movable) {
    // 极简删除：直接断链退链表
    untreeify((TreeNode<K, V>)this);
}

// ===== 通用方法不变 =====
public boolean containsKey(Object key) {
    return getNode(hash(key), key) != null;
}

public int size() {
    return size;
}

public boolean isEmpty() {
    return size == 0;
}

public void clear() {
    Node<K, V>[] tab;
    if ((tab = table) != null && size > 0) {
        size = 0;
        Arrays.fill(tab, null);
    }
}

public Set<K> keySet() {
    Set<K> set = new HashSet<>();
    Node<K, V>[] tab = table;

```



```

        if (tab == null) return set;
        for (Node<K, V> e : tab) {
            for (; e != null; e = e.next)
                set.add(e.key);
        }
        return set;
    }

    public Collection<V> values() {
        List<V> list = new ArrayList<>();
        Node<K, V>[] tab = table;
        if (tab == null) return list;
        for (Node<K, V> e : tab) {
            for (; e != null; e = e.next)
                list.add(e.value);
        }
        return list;
    }

    public Set<Map.Entry<K, V>> entrySet() {
        Set<Map.Entry<K, V>> set = new HashSet<>();
        Node<K, V>[] tab = table;
        if (tab == null) return set;
        for (Node<K, V> e : tab) {
            for (; e != null; e = e.next)
                set.add(new AbstractMap.SimpleEntry<>(e.key, e.value));
        }
        return set;
    }

    // ===== 测试 Main 函数 =====
    public static void main(String[] args) {
        MyHashMap<String, Integer> map = new MyHashMap<>();

        // 1. 插入
        map.put("语文", 88);
        map.put("数学", 95);
        map.put("英语", 90);
        System.out.println("【插入后元素个数】: " + map.size());

        // 2. 查找
        System.out.println("【查询数学】: " + map.get("数学"));

        // 3. 修改(覆盖)
        map.put("数学", 100);
        System.out.println("【修改后数学】: " + map.get("数学"));

        // 4. 判断key是否存在
        System.out.println("【是否包含英语】: " + map.containsKey("英语"));
        System.out.println("【是否包含体育】: " + map.containsKey("体育"));
    }

```

```
// 5. 遍历key
System.out.print("\n【遍历所有Key】：");
for (String key : map.keySet()) {
    System.out.print(key + " ");
}

// 6. 遍历value
System.out.print("\n【遍历所有Value】：");
for (Integer val : map.values()) {
    System.out.print(val + " ");
}

// 7. 遍历键值对
System.out.println("\n\n【遍历所有键值对】：");
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " : " + entry.getValue());
}

// 8. 删除
map.remove("英语");
System.out.println("\n【删除英语后元素个数】： " + map.size());

// 9. 清空
map.clear();
System.out.println("【清空后元素个数】： " + map.size());
System.out.println("【是否为空】： " + map.isEmpty());
}
}
```